Changwe Musonda

Yucaipa, CA | +1 (909) 312-8519 | bradley.musonda@gmail.com | LinkedIn | GitHub | Website

Education

California State Polytechnic University, Pomona Aug 2026 – Dec 2027 (Expected)
Master of Science in Electrical Engineering

California State Polytechnic University, Pomona May 2026
Bachelor of Science in Computer Engineering *GPA: 3.56*

Crafton Hills College May 2023
Associate of Science in Mathematics *GPA: 3.67*

Work Experience

Autonomous Vehicle Laboratory Pomona, CA
Research Assistant *May 2025 – Present*

- Developed real-time **C++** and **Python** software for localization, mapping, and control on **ARM**-based embedded Linux systems within **ROS 2**-based autonomous platforms operating in real-world conditions
- Integrated **LiDAR**, **ZED X stereo cameras** (GMSL), and **Xsens IMUs** in multi-sensor state estimation pipelines; performed driver-level bring-up, calibration, and sensor fusion parameter tuning to improve localization robustness
- Diagnosed and resolved failures across distributed **ROS 2** systems spanning hardware interfaces, sensor timing, and node communication during lab integration and field testing
- Produced architecture design, calibration procedures, and validation workflow documentation to support reproducibility and team-wide integration

Team Pegasus (Lockheed Martin Sponsored) Pomona, CA
Autonomy Engineer → Autonomy Lead (Senior Capstone) *Jan 2025 – May 2026*

- Led a team of 7 engineers to design and integrate an end-to-end UAV autonomy system in **C++** and **Python** on **ARM**-based embedded Linux (Jetson AGX), progressing from **Gazebo SITL** simulation through **hardware-in-the-loop** testing to outdoor field deployment
- Bridged **PX4** flight controller firmware to **ROS 2** via **uXRCE-DDS**; integrated **LiDAR** and **ZED X stereo cameras** (GMSL bring-up) into a unified perception pipeline supporting **SLAM**-based localization and closed-loop control
- Diagnosed hardware and software integration failures using telemetry logs and root-cause analysis, tracing faults across firmware, OS-level timing, and application software to improve reliability from lab through outdoor flight test
- Collaborated with cross-functional teams on system integration, test readiness reviews, and iterative validation; produced wiring diagrams, calibration procedures, and integration documentation

Projects

Custom Flight/Motor Controller Firmware (STM32, FreeRTOS) Aug 2026 – Present

- Designing a bare-metal flight controller firmware stack on an **STM32** Nucleo board using **FreeRTOS**, with a custom **SPI/I2C** IMU driver written from register-level configuration, explicitly avoiding vendor HAL abstractions to build genuine low-level embedded competency
- Architecting an interrupt-driven control loop: sensor read → attitude estimation filter → **PWM** actuator output, alongside a watchdog timer and multi-mode **fault-handling** task implemented at the firmware and interrupt level
- Planning a **UART/CAN** telemetry link for real-time logging and a bench validation campaign measuring loop timing, filter convergence behavior, and fault-injection response on a live motor/ESC rig

Project DOLPHIN — AUV Swarm Autonomy Jan 2026 – Present

- Designed a **behavior tree** autonomy architecture in **C++17** running at 10 Hz on a **Raspberry Pi 5**, commanding a **Pixhawk 6X** via **MAVSDK** with hard real-time interface and timing requirements
- Architected a five-mode **fault handling** system covering flight controller link loss, battery management, leak detection, perception degradation, and acoustic comms loss, each with defined autonomous recovery policies
- Defined the compute interface across a three-layer stack (Pi 5 → Pixhawk → **Jetson Orin Nano**), with **confidence gating** on perception output to prevent faulty detections from driving control commands
- Designed a comms-loss policy enabling **fully autonomous operation** without inter-vehicle communication, treating acoustic link loss as expected operating behavior rather than a fault condition

Deep Differentiable Neural Planner for Egocentric Navigation Jan 2026 – May 2026

- Trained and benchmarked four differentiable planning architectures (**VIN**, **GPPN**, CALVINConv2d, CALVINConv3d) on custom egocentric and allocentric datasets, extending a CVPR 2022 paper to evaluate cross-dataset generalization
- Modernized a legacy **PyTorch** research codebase, resolving GPU/CPU tensor handling and CUDA dependency issues to establish a reproducible ML experiment pipeline

Broncos Gambit Chess Engine Jan 2026 – May 2026

- Developed a fully functional chess engine in **C** as part of a 5-person team managed with **Jira** and **Git**, communicating over the **UCI** protocol for plug-in compatibility with any standard chess GUI
- Implemented **alpha-beta search** with negamax, quiescence search, and a 1M-entry **Zobrist-hashed transposition table** (32 MB); applied iterative deepening and move ordering heuristics to improve pruning efficiency

Unmanned Ground Vehicle SLAM System Dec 2025 – Present

- Developed a **ROS 2**-based **SLAM** pipeline integrating mapping and localization for autonomous navigation in unstructured outdoor environments
- Improved localization robustness by analyzing failure cases under sensor noise and environmental variation, refining pipeline parameters through iterative field testing

A* Path Planning on FPGA (Verilog + Python) Aug 2025 – Dec 2025

- Implemented a hardware-accelerated A* path planning algorithm in **Verilog** on an **FPGA**, enabling parallel execution and exposing hardware/software co-design tradeoffs between FPGA execution and software-side orchestration
- Validated hardware output against **Python** reference models across structured test scenarios, performing performance analysis to measure execution time improvements and resource utilization

Technical Skills

Languages: C/C++ (C++17), Python, Verilog
Tools: ROS 2 (Humble), PX4, uXRCE-DDS, MAVSDK, Pixhawk, PyTorch, Git, Linux
Systems: ARM Embedded Linux (Jetson AGX, Jetson Orin Nano, Raspberry Pi), SLAM, Sensor Fusion, Behavior Tree Autonomy, Fault Handling, Field Deployment & Validation